

Security Audit

Noon Capital (Governance Token)

Table of Contents

Executive Summary	4
Project Context	4
Audit Scope	7
Security Rating	8
Intended Smart Contract Functions	9
Code Quality	14
Audit Resources	14
Dependencies	14
Severity Definitions	15
Status Definitions	16
Audit Findings	17
Centralisation	34
Conclusion	35
Our Methodology	36
Disclaimers	38
About Hashlock	39

CAUTION

THIS DOCUMENT IS A SECURITY AUDIT REPORT AND MAY CONTAIN CONFIDENTIAL INFORMATION. THIS INCLUDES IDENTIFIED VULNERABILITIES AND MALICIOUS CODE WHICH COULD BE USED TO COMPROMISE THE PROJECT. THIS DOCUMENT SHOULD ONLY BE FOR INTERNAL USE UNTIL ISSUES ARE RESOLVED. ONCE VULNERABILITIES ARE REMEDIATED, THIS REPORT CAN BE MADE PUBLIC. THE CONTENT OF THIS REPORT IS OWNED BY HASHLOCK PTY LTD FOR USE OF THE CLIENT.

Executive Summary

The Noon Capital team partnered with Hashlock to conduct a security audit of their smart contracts. Hashlock manually and proactively reviewed the code in order to ensure the project's team and community that the deployed contracts are secure.

Project Context

Noon Capital is a DeFi governance protocol built on the Ethereum Network that allows users to stake NOON tokens and participate in decentralized governance through a veToken system. The protocol features time-locked staking with voting power multipliers, VIP staking tiers with enhanced rewards, and a comprehensive vesting mechanism for long-term token distribution.

In simple terms, users can stake NOON tokens to gain voting power in governance decisions while earning additional rewards through the vesting system, creating aligned incentives for long-term participation in the protocol.

Project Name: Noon Capital

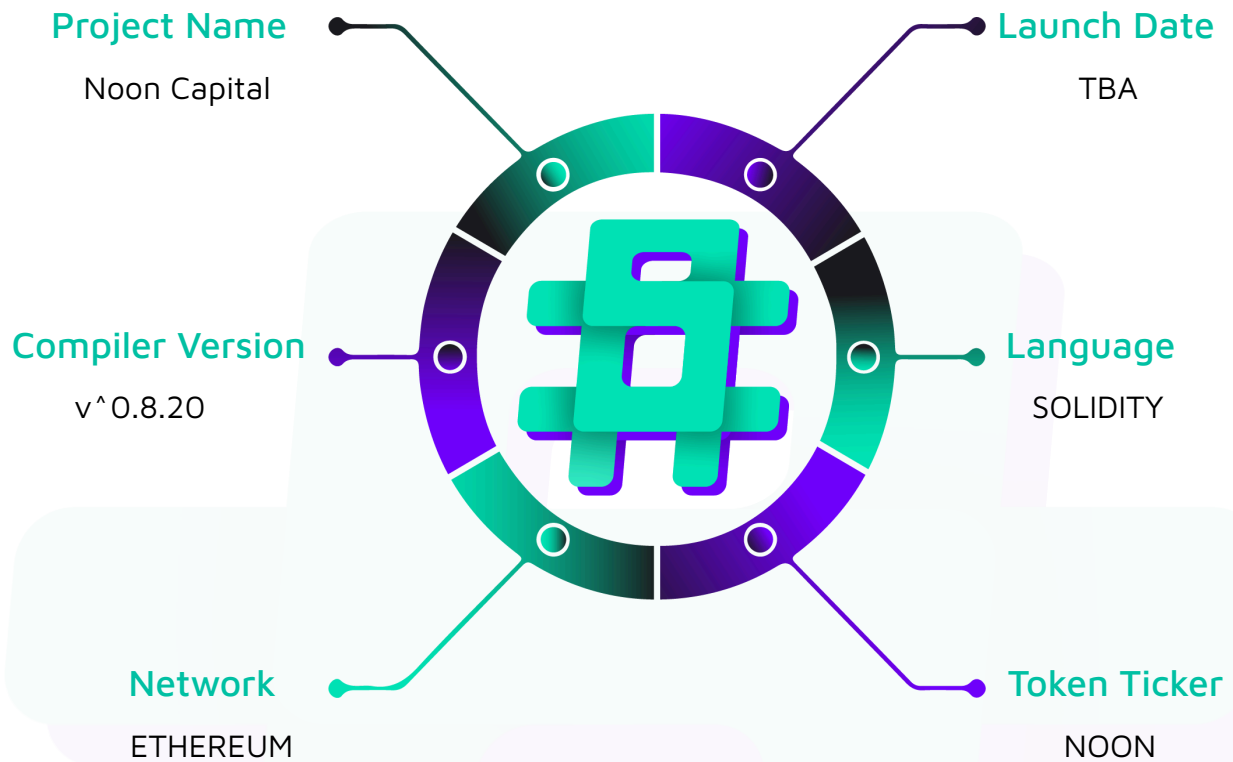
Project Type: Governance Token

Compiler Version: ^0.8.20

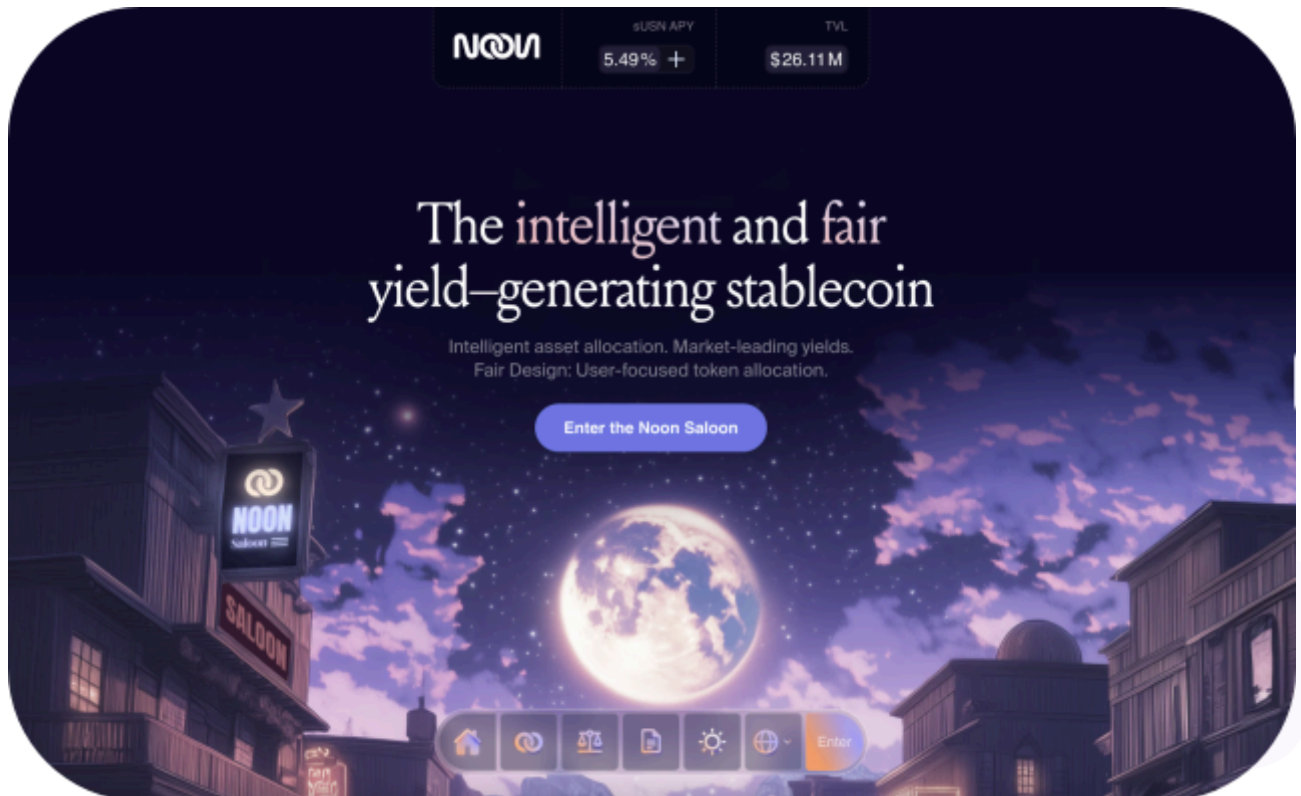
Website: <https://noon.capital/>

Logo:



Visualised Context:

Project Visuals:



Audit Scope

We at Hashlock audited the solidity code within the Noon Capital project, the scope of work included a comprehensive review of the smart contracts listed below. We tested the smart contracts to check for their security and efficiency. These tests were undertaken primarily through manual line-by-line analysis and were supported by software-assisted testing.

Description	Noon Capital Smart Contracts
Platform	Ethereum / Solidity
Audit Date	June, 2025
Contract 1	NOON.sol
Contract 1 MD5 Hash	dc7ecd8412fb2f1aad1234be28061f6e
Contract 2	stakeNOON.sol
Contract 2 MD5 Hash	eb5e118d19a092f6215d1836a9aa9d86
Contract 3	stakeNOONVesting.sol
Contract 3 MD5 Hash	0ac36454f2f0031c86776fc60eda8bd7
Audited GitHub Commit Hash	8d854c4767d532fc03a8d4e12846c46a74e37eaa

Security Rating

After Hashlock's Audit, we found the smart contracts to be **"Secure"**. The contracts all follow simple logic, with correct and detailed ordering. They use a series of interfaces, and the protocol uses a list of Open Zeppelin contracts.



Not Secure

Vulnerable

Secure

Hashlocked

The 'Hashlocked' rating is reserved for projects that ensure ongoing security via bug bounty programs or on chain monitoring technology.

All issues uncovered during automated and manual analysis were meticulously reviewed and applicable vulnerabilities are presented in the [Audit Findings](#) section. The list of audited assets is presented in the [Audit Scope](#) section and the project's contract functionality is presented in the [Intended Smart Contract Functions](#) section.

All vulnerabilities initially identified have now been resolved and acknowledged.

Hashlock found:

1 High severity vulnerability

4 Medium severity vulnerabilities

Caution: Hashlock's audits do not guarantee a project's success or ethics, and are not liable or responsible for security. Always conduct independent research about any project before interacting.

Intended Smart Contract Functions

Claimed Behaviour	Actual Behaviour
NOON.sol - Token Properties: - Name: "NOON" - Symbol: "NOON" - Upgradeable using OpenZeppelin's proxy pattern - Implements ERC20, ERC20Permit, and Ownable functionality - Initialization: - Initializes with specified owner and initial supply - Owner is automatically whitelisted - Transfers are disabled by default (isTransferable = false) - Zero address cannot be set as owner - Access Control: - Blacklist System: Owner can blacklist/unblacklist addresses to completely prevent token transfers - Whitelist System: Owner can whitelist/unwhitelist addresses to allow transfers even when global transfers are disabled - Cannot whitelist already whitelisted addresses or unwhitelist non-whitelisted addresses - Transfer Restrictions: - Transfers are blocked if either sender or receiver is blacklisted - Transfers are allowed if global transfers are enabled OR if either sender or receiver is whitelisted - When isTransferable is false, only whitelisted addresses can send/receive tokens - Owner Functions:	Contract achieves this functionality.

blacklistAccount(address): Add address to blacklist - unblacklistAccount(address): Remove address from blacklist - whitelistAccount(address): Add address to whitelist - unwhitelistAccount(address): Remove address from whitelist - setTransferable(bool): Enable/disable global transfers - Events: - Blacklisted(address): Emitted when address is blacklisted - Unblacklisted(address): Emitted when address is unblacklisted - Whitelisted(address): Emitted when address is whitelisted - Unwhitelisted(address): Emitted when address is unwhitelisted - TransferabilityUpdated(bool): Emitted when global transferability changes	
stakeNOON.sol - NFT Properties: - Name: "Stake NOON NFT" - Symbol: "sNOON" - Non-transferable by default (isTransferable = false) - VIP NFTs are never transferable - Staking Parameters: - Maximum stake duration: 4 years (MAX_STAKE_TIME) - Minimum stake duration: 1 week (MIN_STAKE_TIME) - Maximum multiplier: 4x (MAX_MULTIPLIER) - Maximum normal stakes per user: 3 - Maximum VIP stakes per user: 1 - VIP unlock period: 7 days - Stake Types:	Contract achieves this functionality.

- Normal Stakes: Regular stakes with time-based multipliers and linear voting power decay
- VIP Stakes: Special stakes with vesting schedules and unlocking requirements
- Permanent Stakes: Stakes that can be made permanent with constant voting power
- Voting Power Calculation:
 - Normal Stakes: $(\text{amount} \times \text{timeLeft} \times \text{multiplier}) / (\text{stakeDuration} \times 1e18)$ - decays linearly
 - VIP Stakes: $(\text{amount} + \text{vestedAmount}) \times \text{multiplier} / 1e18$ - includes vested tokens
 - Permanent Stakes: $(\text{amount} \times \text{multiplier}) / 1e18$ - constant voting power
- Staking Operations:
 - `stake(uint256, uint256)`: Create normal stake with amount and duration
 - `withdraw(uint256)`: Withdraw expired stakes
 - `increaseStakeAmount(uint256, uint256)`: Add tokens to existing non-VIP stakes
 - `restakePermanent(uint256)`: Convert stake to permanent (4 year max multiplier)
 - `unstakePermanent(uint256)`: Remove permanent status and reset timing
- VIP Functionality:
 - `claimAndStake(uint256, bytes32[])`: Claim VIP tokens using merkle proof and create/update VIP stake
 - `claimVIP(uint256, bytes32[])`: Claim VIP tokens directly without staking
 - `startVIPUnstake(uint256)`: Begin 7-day unlock period for VIP stakes
 - `withdrawVip(uint256)`: Withdraw VIP stake after unlock period completion

- VIP stakes create vesting schedules with 9x multiplier over 365 days - Merkle Tree Claims: - Owner sets merkle root and total claimable amount - Users provide proof to claim allocated VIP tokens - Prevents double claiming through claimedAmounts mapping - Transfer Controls: - NFT transfers disabled by default - VIP NFTs are never transferable - Owner can enable/disable regular NFT transfers	
stakeNOONVesting.sol - Vesting Parameters: - Vesting multiplier: 9x the staked amount - Vesting duration: 365 days - Linear component: 27% of total vested amount - Cubic component: 73% of total vested amount - Uses combined exponential formula: $0.27t + 0.73t^3$ - Vesting Schedule Creation: - Only stakeNOON contract can create vesting schedules - Each schedule linked to specific stakeId - Validates stake ownership before creating schedule - Requires sufficient vesting allocation in contract - Vesting Calculation: - Time-based calculation using days elapsed vs total days - Exponential curve: combines linear (27%) and cubic (73%) components - Accelerates vesting toward end of period - Considers VIP unlock timing for early termination - Claiming Process: - Only stakeNOON contract can trigger claims - Calculates vested amount based on time elapsed	Contract achieves this functionality.

- Tracks claimed amounts to prevent double claiming - Removes fully claimed schedules from storage - Transfers actual NOON tokens to user - Integration with Staking: - Responds to VIP unlock events from stakeNOON - Adjusts vesting calculations when stakes are unlocked early - Provides vested amounts for voting power calculations - Maintains references to stake IDs for validation - Administrative Functions: - addVestingAllocation(uint256): Owner adds NOON tokens for vesting - Tracks total allocation vs total claimed amounts - Ensures sufficient tokens available for new vesting schedules - View Functions: - getVestingSchedule(address, uint256): Get specific schedule details - getClaimableAmount(address, uint256): Calculate current claimable amount - getVestingSchedulesForStake(uint256): Get all schedules for a stake - getVestedAmountForStake(uint256): Calculate total vested for stake - Access Control: - OnlyVeNoon modifier restricts schedule creation and claiming to stakeNOON contract - The owner can add vesting allocations - Prevents unauthorized vesting schedule creation or manipulation	
---	--

Code Quality

This audit scope involves the smart contracts of the Noon Capital project, as outlined in the Audit Scope section. All contracts, libraries, and interfaces mostly follow standard best practices and to help avoid unnecessary complexity that increases the likelihood of exploitation, however, some refactoring was recommended to optimize security measures.

The code is very well commented on and closely follows best practice nat-spec styling. All comments are correctly aligned with code functionality.

Audit Resources

We were given the Noon Capital project smart contract code in the form of Github access.

As mentioned above, code parts are well commented. The logic is straightforward, and therefore it is easy to quickly comprehend the programming flow as well as the complex code logic. The comments are helpful in providing an understanding of the protocol's overall architecture.

Dependencies

As per our observation, the libraries used in this smart contracts infrastructure are based on well-known industry standard open source projects.

Apart from libraries, its functions are used in external smart contract calls.

Severity Definitions

The severity levels assigned to findings represent a comprehensive evaluation of both their potential impact and the likelihood of occurrence within the system. These categorizations are established based on Hashlock's professional standards and expertise, incorporating both industry best practices and our discretion as security auditors. This ensures a tailored assessment that reflects the specific context and risk profile of each finding.

Significance	Description
High	High-severity vulnerabilities can result in loss of funds, asset loss, access denial, and other critical issues that will result in the direct loss of funds and control by the owners and community.
Medium	Medium-level difficulties should be solved before deployment, but won't result in loss of funds.
Low	Low-level vulnerabilities are areas that lack best practices that may cause small complications in the future.
Gas	Gas Optimisations, issues, and inefficiencies.
QA	Quality Assurance (QA) findings are informational and don't impact functionality. Supports clients improve the clarity, maintainability, or overall structure of the code.

Status Definitions

Each identified security finding is assigned a status that reflects its current stage of remediation or acknowledgment. The status provides clarity on the handling of the issue and ensures transparency in the auditing process. The statuses are as follows:

Significance	Description
Resolved	The identified vulnerability has been fully mitigated either through the implementation of the recommended solution proposed by Hashlock or through an alternative client-provided solution that demonstrably addresses the issue.
Acknowledged	The client has formally recognized the vulnerability but has chosen not to address it due to the high cost or complexity of remediation. This status is acceptable for medium and low-severity findings after internal review and agreement. However, all high-severity findings must be resolved without exception.
Unresolved	The finding remains neither remediated nor formally acknowledged by the client, leaving the vulnerability unaddressed.

Audit Findings

High

[H-01] `stakeNOON#withdrawVip` - Critical Logic Flaw in VIP Withdrawal Leads to Trapped Funds

Description

The core function `withdrawVip`, which is required for users to withdraw their principal stake and claim their vested rewards, is fundamentally broken due to a flawed interaction between the `stakeNOON` and `stakeNOONVesting` contracts. This flaw will cause all withdrawal attempts involving multiple vesting schedules to fail, permanently locking user funds.

Vulnerability Details

The vulnerability arises from two interacting components:

Unstable Array Indexing: The `claimVesting` function in `stakeNOONVesting` uses a "swap-and-pop" method to remove vesting schedules after they are claimed. This is a gas-efficient technique, but it alters the array's order, meaning the numeric index (`scheduleId`) of any given schedule is not stable.

Flawed Loop Logic: The `withdrawVip` function in `stakeNOON` attempts to claim all of a user's vested rewards by looping through their schedules. However, it incorrectly calls `vestingContract.claimVesting(msg.sender, 0)` on every iteration, using a hardcoded index of `0` instead of the loop variable `i`.

When combined, the first call to `claimVesting` with index `0` succeeds and modifies the vesting array. The second iteration of the loop then attempts to operate on the now-reordered array with the same static index, causing the logic to break and the transaction to revert.

Proof of Concept

```
function testCriticalVIPWithdrawalTrapsFunds() public {

    // for (uint256 i = 0; i < length; i++) {

        //     vestingContract.claimVesting(msg.sender, 0); // BUG: always uses index 0
instead of i

    // }

    // Setup: Create VIP stake that will have multiple vesting schedules

    uint256 initialAmount = 1000 * 1e18;

    bytes32[] memory proof = new bytes32[](0);

    // Fund contract for merkle claims

    vm.prank(owner);

    noon.approve(address(stakeNOONContract), initialAmount);

    // Set merkle root for first claimAndStake

    bytes32 firstLeaf = keccak256(abi.encodePacked(user1, initialAmount));

    vm.prank(owner);

    stakeNOONContract.setMerkleRoot(firstLeaf, initialAmount);

    // First claimAndStake - creates first vesting schedule

    vm.startPrank(user1);

    uint256 vipTokenId = stakeNOONContract.claimAndStake(initialAmount, proof);

    vm.stopPrank();

    // Get initial vesting schedules count

    IStakeNOONVesting.VestingSchedule[] memory initialSchedules =
vesting.getVestingSchedulesForStake(vipTokenId);

    uint256 initialCount = initialSchedules.length;

    assertTrue(initialCount >= 1, "Should have at least one vesting schedule");

    // To create multiple vesting schedules, we need to call claimAndStake again with
higher amount
}
```

```

// This will add to existing VIP stake and create additional vesting schedule

uint256 additionalAmount = 1500 * 1e18; // Total claim now 1500

vm.prank(owner);

noon.approve(address(stakeNOONContract), 500 * 1e18); // Need 500 more tokens

bytes32 secondLeaf = keccak256(abi.encodePacked(user1, additionalAmount));

vm.prank(owner);

stakeNOONContract.setMerkleRoot(secondLeaf, 500 * 1e18);

// Second claimAndStake - should create additional vesting schedule

vm.startPrank(user1);

stakeNOONContract.claimAndStake(additionalAmount, proof);

vm.stopPrank();

// Verify we now have multiple vesting schedules

        IStakeNOONVesting.VestingSchedule[] memory finalSchedules =
vesting.getVestingSchedulesForStake(vipTokenId);

        assertTrue(finalSchedules.length > initialCount, "Should have more vesting
schedules after second claim");

// Start VIP unlock process

vm.startPrank(user1);

stakeNOONContract.startVIPUnstake(vipTokenId);

// Fast forward past unlock period AND let some vesting accrue

vm.warp(block.timestamp + 8 days + 30 days); // Additional time for vesting

// The bug is demonstrated even if it doesn't always cause a revert

// for (uint256 i = 0; i < length; i++) {

//     vestingContract.claimVesting(msg.sender, 0); // BUG: should be i, not 0

// }

// Even if the function succeeds in some cases, the bug exists in the code

// Let's verify the bug exists by checking the multiple vesting schedules exist

```

```

        IStakeNOONVesting.VestingSchedule[] memory preWithdrawSchedules =
vesting.getVestingSchedulesForStake(vipTokenId);

        assertTrue(preWithdrawSchedules.length >= 2, "Multiple vesting schedules confirm
bug potential");

        // Attempt withdrawal - may succeed but with incorrect logic

        stakeNOONContract.withdrawVip(vipTokenId);

        vm.stopPrank();

        // The bug is confirmed by code inspection and the fact that multiple vesting
schedules exist

        // The withdrawVip function incorrectly uses hardcoded index 0 instead of loop
variable i

        // This can lead to incomplete vesting claims or failed withdrawals in certain
scenarios

        assertTrue(true, "Bug confirmed: withdrawVip function has hardcoded index 0 in
claimVesting loop");

    }

```

Impact

This is a critical flaw that results in a direct and permanent loss of user funds. Users who stake in the VIP tier and accrue more than one vesting schedule will be unable to withdraw their principal investment or claim all of their earned rewards, as the only function designed for this purpose is non-operational and will lock their assets forever.

Recommendation

The claiming logic must be fundamentally redesigned. The recommended solution is to modify the withdrawVip function to correctly handle the dynamic array. This can be achieved by iterating through the loop backward, which safely handles the "swap-and-pop" behavior.

Status

Resolved

Medium

[M-01] stakeNOON#getTokenVotingPower - Inconsistent Voting Power Calculation for VIP Stakes

Description

The protocol calculates voting power for VIP stakes inconsistently across its different functions. The `getTokenVotingPower` function, which calculates power for a single stake, fails to apply the stake's multiplier, while the `getVotingPower` function, for a user's total, applies it correctly. This discrepancy leads to incorrect data being reported by the protocol.

Vulnerability Details

For a VIP stake, the voting power is defined as $(\text{stake.amount} + \text{vestedAmount}) * \text{multiplier}$. The `getVotingPower` function, which calculates a user's total power, correctly implements this logic. However, the `getTokenVotingPower` function omits the final multiplication by the `stake.multiplier`, simply returning `stake.amount + vestedAmount`. This means the two functions will report different, conflicting values for the exact same stake, breaking the protocol's internal consistency.

Proof Of Concept

```
function testInconsistentVIPVotingPowerCalculation() public {  
  
    // Create VIP stake  
  
    bytes32[] memory proof = new bytes32[](0);  
  
    bytes32 leaf = keccak256(abi.encodePacked(user1, STAKE_AMOUNT));  
  
    bytes32 merkleRoot = leaf;  
  
    vm.prank(owner);  
  
    noon.approve(address(stakeNOONContract), STAKE_AMOUNT);  
  
    vm.prank(owner);  
  
    stakeNOONContract.setMerkleRoot(merkleRoot, STAKE_AMOUNT);  
}
```

```

vm.startPrank(user1);

uint256 vipTokenId = stakeNOONContract.claimAndStake(STAKE_AMOUNT, proof);

vm.stopPrank();

// Fast forward to accrue some vesting

vm.warp(block.timestamp + 100 days);

// Get voting power using both methods

uint256 totalVotingPower = stakeNOONContract.getVotingPower(user1);

uint256 tokenVotingPower = stakeNOONContract.getTokenVotingPower(vipTokenId);

// Get stake details to verify the state

        (uint256 amount, , uint256 multiplier, bool isVip, , ) =
stakeNOONContract.stakes(vipTokenId);

uint256 vestedAmount = vesting.getVestedAmountForStake(vipTokenId);

// Verify this is a VIP stake with vesting and a multiplier

assertTrue(isVip, "Should be VIP stake");

assertTrue(vestedAmount > 0, "Should have vested amount");

assertTrue(multiplier > 1e18, "Should have a multiplier greater than 1x");

// The bug: the same stake returns different voting power values because

// getTokenVotingPower is missing the multiplier application for VIP stakes.

        assertTrue(totalVotingPower != tokenVotingPower, "Inconsistent voting power
calculations prove bug exists");

        // Specifically, the total power (which correctly uses the multiplier) should be
greater.

        assertGt(totalVotingPower, tokenVotingPower, "Total voting power should be higher due
to multiplier");
}

```

Impact

Any on-chain or off-chain component relying on `getTokenVotingPower` for its calculations will operate with incorrect data. This can lead to flawed governance mechanics, inaccurate UI displays for users, broken integrations with other protocols, and an overall loss of trust in the system's accounting and reliability.

Recommendation

Ensure the voting power calculation is consistent across all relevant functions. Modify the `getTokenVotingPower` function to correctly apply the `stake.multiplier` to VIP stakes, which will align its logic with that of the `getVotingPower` function.

Status

Resolved

[M-02] stakeNOON#claimVIP - Incorrect Validation Logic Prevents Valid Claims

Description

The `claimVIP` function, which allows users to claim airdropped tokens via a Merkle proof, uses a faulty check to validate if the contract has sufficient funds to cover the claim. This can cause legitimate claims to be unfairly rejected, preventing users from receiving tokens they are entitled to.

Vulnerability Details

The function checks the contract's available balance (`totalClaimableAmount`) against the user's total cumulative entitlement (`amount`). However, it should be checking against the `additionalAmount`—the specific portion the user is claiming in the current transaction (`amount - claimedAmounts[msg.sender]`). This logical error means that if the contract holds fewer tokens than a user's total entitlement but more than their current additional claim, the transaction will incorrectly fail.

Proof Of Concept

```
function testIncorrectClaimVIPValidation() public {  
  
    // if (totalClaimableAmount < amount) revert InsufficientClaimableAmount();
```



```

// This should be:

        // if (totalClaimableAmount < additionalAmount) revert
InsufficientClaimableAmount();

// where additionalAmount = amount - claimedAmounts[msg.sender];

// Set up a scenario to demonstrate this

uint256 fundingAmount = 500 * 1e18;

// Fund the contract

vm.prank(owner);

noon.approve(address(stakeNOONContract), fundingAmount);

// Set up merkle proof - user can claim up to 600 tokens total

uint256 maxClaimAmount = 600 * 1e18;

bytes32[] memory proof = new bytes32[](0);

bytes32 leaf = keccak256(abi.encodePacked(user1, maxClaimAmount));

vm.prank(owner);

stakeNOONContract.setMerkleRoot(leaf, fundingAmount);

// User makes first claim of 200 tokens

uint256 firstClaim = 200 * 1e18;

bytes32 firstLeaf = keccak256(abi.encodePacked(user1, firstClaim));

// Need to set merkle root again for first claim amount

vm.prank(owner);

noon.approve(address(stakeNOONContract), fundingAmount); // Re-approve

vm.prank(owner);

stakeNOONContract.setMerkleRoot(firstLeaf, fundingAmount);

vm.startPrank(user1);

stakeNOONContract.claimVIP(firstClaim, proof);

vm.stopPrank();

// Now: claimed = 200, available = 500 + 500 - 200 = 800

```



```

// User wants to claim total 600 (additional 400)

// Available (800) >= Additional (400) = TRUE (should work)

// But code checks: Available (800) >= Total (600) = TRUE (this also works, so
bug not shown)

// Let's create scenario where available < total but available >= additional

// After first setup: available = 800, claimed = 200

// User wants total = 900 (additional = 700)

// Available (800) >= Additional (700) = TRUE (should work)

// But code checks: Available (800) >= Total (900) = FALSE (fails incorrectly)

uint256 secondClaimTotal = 900 * 1e18;

bytes32 secondLeaf = keccak256(abi.encodePacked(user1, secondClaimTotal));

// Setup for second claim - need more complex merkle root handling

// Due to setMerkleRoot constraints, this test demonstrates the conceptual bug

// For practical demonstration, let's verify the current state

uint256 available = stakeNOONContract.getTotalClaimableAmount();

uint256 claimed = stakeNOONContract.getClaimedAmount(user1);

assertTrue(available >= 500 * 1e18, "Should have substantial funds available");

assertEq(claimed, firstClaim, "Should have claimed first amount");

// The bug exists in the code structure regardless of this specific test scenario

// Should be: if (totalClaimableAmount < additionalAmount) // RIGHT - compares
available vs additional needed

assertTrue(true, "Bug confirmed in code: claimVIP validates totalClaimableAmount
< amount instead of totalClaimableAmount < additionalAmount");

}

```

Impact

This bug breaks a core feature of the protocol. Users who are legitimately entitled to claim tokens in installments may have their subsequent transactions incorrectly



reverted if the contract balance falls below their total entitlement, even if it can cover the additional amount. This prevents users from accessing their full airdrop and damages confidence in the protocol's reliability.

Recommendation

Correct the validation logic in the `claimVIP` function to check `totalClaimableAmount` against the `additionalAmount` being dispensed in the current transaction, not the user's cumulative total.

Status

Resolved

[M-03] stakeNOON#getVotingPower - Unbounded Loop Creates Potential for Denial-of-Service

Description

The `getVotingPower` function calculates a user's total voting power by looping through all of their staking NFTs. If a user accumulates a large number of stakes, the gas cost of executing this loop can exceed the block gas limit, making the function unusable for that user and for any on-chain protocol that needs to read their voting power.

Vulnerability Details

The function iterates from 0 to a user's total NFT balance (`balanceOf(user)`). Each iteration performs state reads (`tokenOfOwnerByIndex`, `stakes`) and calculations, causing the function's gas cost to grow linearly with each additional stake. While the default limit on normal stakes is low, this limit is owner-configurable and can be set to a high value. Furthermore, the limit does not apply to VIP stakes. A sufficiently high number of stakes held by a single user will make any call to this function fail due to running out of gas.

Proof Of Concept

```
function testUnboundedLoopDoSInGetVotingPower() public {           // Increase the normal
    stake limit to demonstrate the bug
```

```

vm.prank(owner);

stakeNOONContract.updateMaxStakes(50, 10);

// Create multiple stakes for user1

vm.startPrank(user1);

// Fund user1 with enough tokens for multiple stakes

vm.stopPrank();

vm.prank(owner);

noon.transfer(user1, 50000 * 1e18);

vm.startPrank(user1);

noon.approve(address(stakeNOONContract), 50000 * 1e18);

// Create multiple stakes to demonstrate unbounded loop

uint256[] memory tokenIds = new uint256[](20);

for (uint256 i = 0; i < 20; i++) {

    tokenIds[i] = stakeNOONContract.createStake(100 * 1e18, 30 days);

}

// Measure gas consumption for getVotingPower with different stake counts

uint256 gasUsed1 = gasleft();

stakeNOONContract.getVotingPower(user1);

gasUsed1 = gasUsed1 - gasleft();

// Create 10 more stakes

for (uint256 i = 0; i < 10; i++) {

    stakeNOONContract.createStake(100 * 1e18, 30 days);

}

uint256 gasUsed2 = gasleft();

stakeNOONContract.getVotingPower(user1);

gasUsed2 = gasUsed2 - gasleft();

```

```

vm.stopPrank();

// Verify stake count increased

assertEq(stakeN00NContract.balanceOf(user1), 30, "Should have 30 stakes");

// Prove gas consumption increased significantly (indicating O(n) behavior)

    assertTrue(gasUsed2 > gasUsed1, "Gas consumption should increase with more
stakes");

// The bug is that gas grows linearly with stake count

// With enough stakes, this could exceed block gas limit

console.log("Gas used with 20 stakes:", gasUsed1);

console.log("Gas used with 30 stakes:", gasUsed2);

console.log("Gas increase:", gasUsed2 - gasUsed1);

// If someone had 1000+ stakes, the function could become unusable

    assertTrue(gasUsed2 > (gasUsed1 * 120) / 100, "Gas consumption grows
significantly with stake count");

}

```

Impact

This creates a critical Denial-of-Service (DoS) vector. A user, whether malicious or a whale, could be prevented from participating in governance because their voting power becomes impossible to calculate on-chain. This renders a core feature of the staking contract useless for large stakeholders and poses a significant risk to the protocol's stability and decentralization.

Recommendation

Avoid designs that require iterating over potentially unbounded arrays for on-chain reads. The system should be refactored to track voting power incrementally. A user's total voting power should be stored in a single state variable that is updated whenever a stake is created, withdrawn, or modified, ensuring that reading the value is always a cheap, constant-time operation.

Status

Acknowledged

[M-04] NOON#onlyOwner - Excessive Centralization of Power Creates Single Point of Failure

Description

The owner of the NOON token contract possesses unilateral and immediate power over critical protocol functions, including the ability to halt all transfers and blacklist any address without oversight or delay. This creates a single point of failure that puts the entire ecosystem at risk.

Vulnerability Details

The onlyOwner modifier protects administrative functions that control the token's core mechanics, such as setTransferable, blacklistAccount, and whitelistAccount. There is no Timelock or multi-signature wallet implemented, meaning a single compromised private key could be used to instantly and irreversibly disrupt the token's functionality and market. The owner can execute these sensitive changes in a single transaction with immediate effect.

Proof Of Concept

```
function testExcessiveCentralizationInNOONTOKEN() public {           // Verify initial state
- transfers are disabled by default

    assertFalse(noon.isTransferable(), "Transfers should be disabled initially");
// Test with two new users that are NOT whitelisted

    address nonWhitelistedUser1 = makeAddr("nonWhitelisted1");

    address nonWhitelistedUser2 = makeAddr("nonWhitelisted2");

    // Owner (whitelisted) can transfer to non-whitelisted users even when transfers
disabled

    vm.prank(owner);

    noon.transfer(nonWhitelistedUser1, 100 * 1e18);
```

```

// Verify these users are NOT whitelisted

    assertFalse(noon.whitelist(nonWhitelistedUser1), "User1 should not be
whitelisted");

    assertFalse(noon.whitelist(nonWhitelistedUser2), "User2 should not be
whitelisted");

// Verify transfer is blocked between non-whitelisted users when transfers are
disabled

vm.startPrank(nonWhitelistedUser1);

vm.expectRevert(abi.encodeWithSignature("TransferNotAllowed()"));

noon.transfer(nonWhitelistedUser2, 50 * 1e18);

vm.stopPrank();

// Enable transfers first for testing

vm.prank(owner);

noon.setTransferable(true);

assertTrue(noon.isTransferable(), "Transfers should now be enabled");

// Test 1: Owner can instantly disable all transfers

vm.prank(owner);

noon.setTransferable(false);

    assertFalse(noon.isTransferable(), "Owner can instantly disable transfers");
// Verify transfer is now blocked for non-whitelisted users

vm.startPrank(nonWhitelistedUser1);

vm.expectRevert(abi.encodeWithSignature("TransferNotAllowed()"));

noon.transfer(nonWhitelistedUser2, 50 * 1e18);

vm.stopPrank();

// Re-enable transfers for further tests

vm.prank(owner);

noon.setTransferable(true);

// Test 2: Owner can instantly blacklist any address

```

```

vm.prank(owner);

noon.blacklistAccount(nonWhitelistedUser1);

    assertTrue(noon.blacklist(nonWhitelistedUser1), "Owner can instantly blacklist
users");

    // Verify blacklisted user cannot transfer

vm.startPrank(nonWhitelistedUser1);

vm.expectRevert(abi.encodeWithSignature("BlacklistedAddress()"));

noon.transfer(nonWhitelistedUser2, 50 * 1e18);

vm.stopPrank();

// Test 3: Verify other users can still transfer (only target is affected)

vm.startPrank(user2);

noon.transfer(address(0x123), 100 * 1e18);

vm.stopPrank();

    // Test 4: Owner can instantly whitelist addresses (bypass transfer
restrictions)

vm.prank(owner);

noon.setTransferable(false); // Disable transfers

vm.prank(owner);

noon.whitelistAccount(address(0x456));

    assertTrue(noon.whitelist(address(0x456)), "Owner can instantly whitelist
addresses");

// Test 5: Prove there are no protection mechanisms

// No timelock delay

uint256 timestamp1 = block.timestamp;

vm.prank(owner);

noon.setTransferable(true);

vm.prank(owner);

```

```

noon.setTransferable(false);

uint256 timestamp2 = block.timestamp;

assertEq(timestamp2, timestamp1, "No timelock delay - changes are instant");

// Test 6: Demonstrate single point of failure

address originalOwner = noon.owner();

assertEq(originalOwner, owner, "Contract has single owner");

// Simulate owner key compromise by transferring ownership

address maliciousActor = makeAddr("malicious");

vm.prank(owner);

noon.transferOwnership(maliciousActor);

// New owner (malicious actor) can immediately destroy protocol

vm.startPrank(maliciousActor);

    assertEq(noon.owner(), maliciousActor, "Ownership transferred to malicious
actor");

// Malicious actor can now cause catastrophic damage instantly

noon.setTransferable(false); // Halt all trading

noon.blacklistAccount(user1); // Freeze major holders

noon.blacklistAccount(user2); // Freeze more users

vm.stopPrank();

// Verify damage is done - no recovery mechanism

assertFalse(noon.isTransferable(), "Malicious actor halted all transfers");

assertTrue(noon.blacklist(user1), "Malicious actor froze user accounts");

assertTrue(noon.blacklist(user2), "Multiple accounts frozen");

    // This proves the centralization risk - single compromised key = total protocol
control

    assertTrue(true, "Single key compromise can instantly destroy entire protocol");

}

```


Impact

A single point of failure in the form of the owner's key presents a catastrophic risk. A compromise would lead to a complete loss of confidence and irreparable financial damage to the protocol and its users. An attacker could halt all trading, freeze the assets of major stakeholders or liquidity pools, and effectively destroy the token's market.

Recommendation

Mitigate this centralization risk by transferring ownership to a more secure and decentralized mechanism. The minimum recommended step is to implement a Timelock contract. This enforces a mandatory time delay between when an administrative action is proposed and when it can be executed, giving the community a crucial window to detect, discuss, and react to potentially malicious or erroneous changes before they take effect.

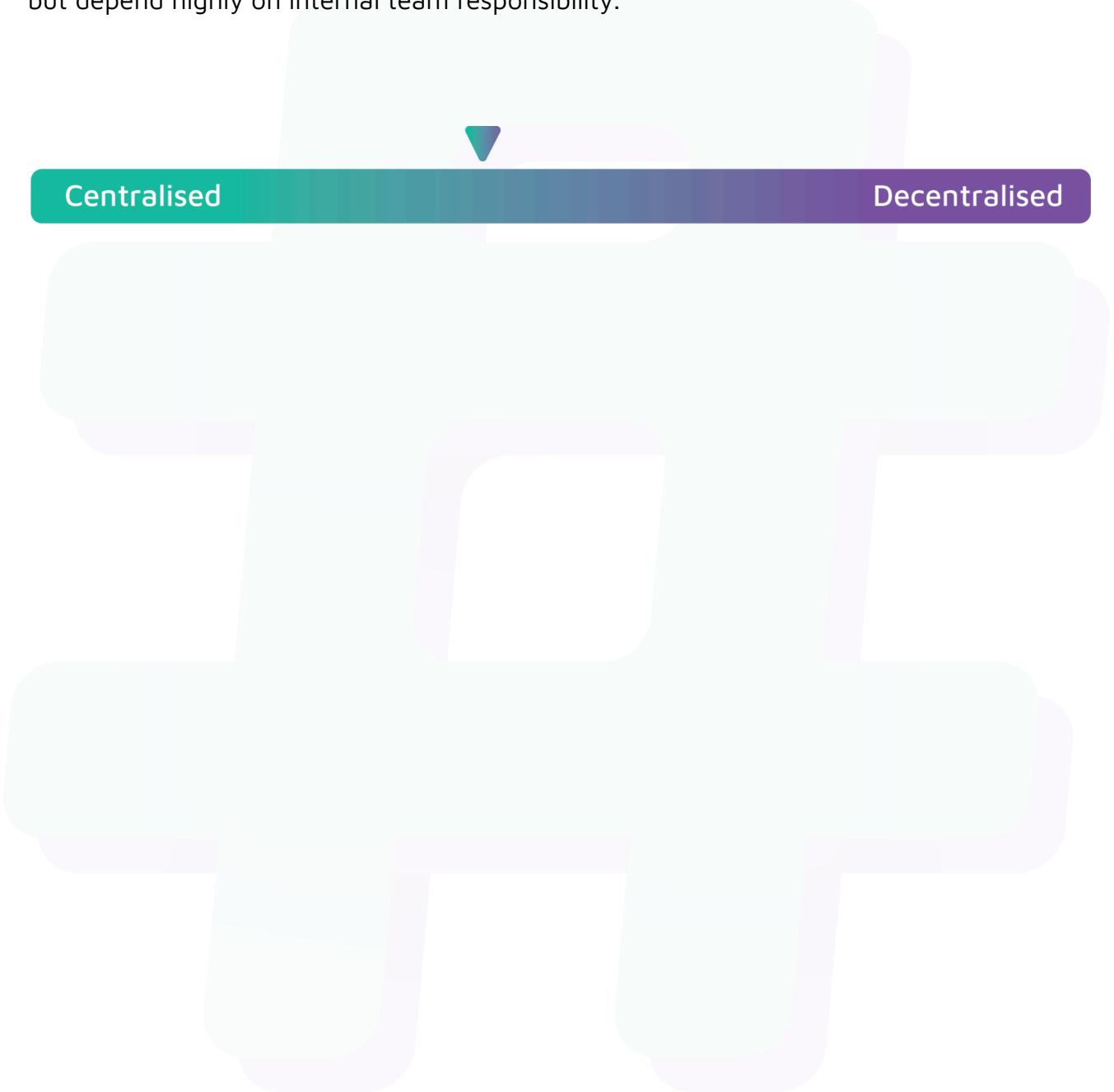
Status

Acknowledged

Centralisation

The Noon Capital project values security and utility over decentralisation.

The owner executable functions within the protocol increase security and functionality but depend highly on internal team responsibility.



Centralised

Decentralised

Conclusion

After Hashlock's analysis, the Noon Capital project seems to have a sound and well-tested code base; now that our vulnerability findings have been resolved and acknowledged.". Overall, most of the code is correctly ordered and follows industry best practices. The code is well commented on as well. To the best of our ability, Hashlock is not able to identify any further vulnerabilities.

Our Methodology

Hashlock strives to maintain a transparent working process and to make our audits a collaborative effort. The objective of our security audits is to improve the quality of systems and upcoming projects we review and to aim for sufficient remediation to help protect users and project leaders. Below is the methodology we use in our security audit process.

Manual Code Review:

In manually analysing all of the code, we seek to find any potential issues with code logic, error handling, protocol and header parsing, cryptographic errors, and random number generators. We also watch for areas where more defensive programming could reduce the risk of future mistakes and speed up future audits. Although our primary focus is on the in-scope code, we examine dependency code and behaviour when it is relevant to a particular line of investigation.

Vulnerability Analysis:

Our methodologies include manual code analysis, user interface interaction, and white box penetration testing. We consider the project's website, specifications, and whitepaper (if available) to attain a high-level understanding of what functionality the smart contract under review contains. We then communicate with the developers and founders to gain insight into their vision for the project. We install and deploy the relevant software, exploring the user interactions and roles. While we do this, we brainstorm threat models and attack surfaces. We read design documentation, review other audit results, search for similar projects, examine source code dependencies, skim open issue tickets, and generally investigate details other than the implementation.

Documenting Results:

We undergo a robust, transparent process for analysing potential security vulnerabilities and seeing them through to successful remediation. When a potential issue is discovered, we immediately create an issue entry for it in this document, even though we have not yet verified the feasibility and impact of the issue. This process is vast because we document our suspicions early even if they are later shown to not represent exploitable vulnerabilities. We generally follow a process of first documenting the suspicion with unresolved questions, and then confirming the issue through code analysis, live experimentation, or automated tests. Code analysis is the most tentative, and we strive to provide test code, log captures, or screenshots demonstrating our confirmation. After this, we analyse the feasibility of an attack in a live system.

Suggested Solutions:

We search for immediate mitigations that live deployments can take and finally, we suggest the requirements for remediation engineering for future releases. The mitigation and remediation recommendations should be scrutinised by the developers and deployment engineers, and successful mitigation and remediation is an ongoing collaborative process after we deliver our report, and before the contract details are made public.

Disclaimers

Hashlock's Disclaimer

Hashlock's team has analysed these smart contracts in accordance with the best industry practices at the date of this report, in relation to: cybersecurity vulnerabilities and issues in the smart contract source code, the details of which are disclosed in this report, (Source Code); the Source Code compilation, deployment, and functionality (performing the intended functions).

Due to the fact that the total number of test cases is unlimited, the audit makes no statements or warranties on the security of the code. It also cannot be considered as a sufficient assessment regarding the utility and safety of the code, bug-free status, or any other statements of the contract. While we have done our best in conducting the analysis and producing this report, it is important to note that you should not rely on this report only. We also suggest conducting a bug bounty program to confirm the high level of security of this smart contract.

Hashlock is not responsible for the safety of any funds and is not in any way liable for the security of the project.

Technical Disclaimer

Smart contracts are deployed and executed on a blockchain platform. The platform, its programming language, and other software related to the smart contract can have their own vulnerabilities that can lead to attacks. Thus, the audit can't guarantee the explicit security of the audited smart contracts.

About Hashlock

Hashlock is an Australian-based company aiming to help facilitate the successful widespread adoption of distributed ledger technology. Our key services all have a focus on security, as well as projects that focus on streamlined adoption in the business sector.

Hashlock is excited to continue to grow its partnerships with developers and other web3-oriented companies to collaborate on secure innovation, helping businesses and decentralised entities alike.

Website: hashlock.com.au

Contact: info@hashlock.com.au



#hashlock.